DOCUMENT

01

Project Overview

What was built, where it stands and where to read more

Purpose

The Invoice Approval app is a small service where invoices are created, submitted and then approved or rejected. This delivery adds **approval limits by role**: how much a person may approve depends on their role, and very large invoices need two different people.

It was built as the home assignment for *AI in the SDLC*: one small feature taken from specification to a reviewed pull request, with the evidence around it (tests, mutation scores, hooks, logs, review verdicts).

The feature in one table

Invoice amount	Who may approve	Approvals needed
up to and including 50,000.00	Manager or Director	1
above 50,000.00, up to and including 500,000.00	Director	1
above 500,000.00	first: Manager or Director; second: **Director**	2 different users

Two rules are deliberately negative and carry most of the risk:

1. **The same user never counts as both approvers.** Usernames are compared after trimming whitespace and ignoring case, so `Dir.One` and `dir.one` are one person.
2. **The second approver above 500,000 must be a Director.** A Manager can give the first approval but never the second.

A refused attempt returns a clear reason (HTTP 403) and **changes nothing**: no status, no decider, no recorded approval.

Status at a glance

Item	Result
Automated tests	177 pytest tests pass (63 policy, 62 service, 36 API, 16 repository contract)
End-to-end	1 Playwright test passes (a manager is refused on a 75,000.00 invoice)
Mutation score (domain package)	before 63.9% (23 of 36 killed), after 100% (132 of 132 killed)
AI code review	11 findings, each with one verdict: 6 fixed, 3 dismissed, 2 deferred
Open items	fork, commit in steps, push and open the PR (nothing is committed yet)

Technology

Area	Choice
Language / framework	Python 3.10+ (built on 3.13), FastAPI, uvicorn
Money	`decimal.Decimal` (never `float`)
Storage	In-memory repository (no database)
UI	Static HTML, CSS and JavaScript served by the API
Unit and API tests	pytest, httpx
Mutation testing	mutmut 3.8 (runs in WSL/Linux, needs `fork()`)
End-to-end tests	Playwright with page objects and role-based locators
Spec-driven workflow	Spec Kit with Claude Code
CI	GitHub Actions (pytest, mutation, e2e)

Why Python, and where the code came from

The upstream project (`NITIN1670/ai-sdlc-demo-app`) is ASP.NET Core 8. The assignment had to be done in Python and the machine has no .NET SDK, so the backend was ported to FastAPI first. The layering (Domain, Api, Tests), the endpoints, the web UI and the Playwright setup were kept, and the 7 original xUnit tests were ported and kept green before any feature work started. That port is the mutation baseline.

Document map

Document	Read it when you want to...
02 Requirements Specification	know exactly what the rules are and how they are accepted
03 Architecture and Design	understand the layers, data model and key decisions
04 API Reference	call the service or read response shapes and error texts
05 User Guide	use the web UI, with screenshots
06 Developer Guide	set up, run, extend and troubleshoot
07 Test Plan and Report	see what is tested and why the tests can fail
08 Mutation Testing Report	see the before and after scores and how survivors were handled
09 Code Review Report	see every review finding and its verdict
10 AI Workflow Guide	see the agents, skills, hooks and interaction log

Document	Read it when you want to...
11 Operations and Security	know the limits, known issues and follow-ups
12 Specification Pack	read the full Spec Kit artifacts (spec, plan, tasks, contract)
13 Assignment Report	see how each assignment item was met
14 Appendix	read the agent conventions (`AGENTS.md`) and the feature briefs for future work
15 Demo Video Script	record the 8 to 10 minute demo with a timed, rehearsed script
16 Submission Checklist	see what is done, what is open and what to watch for
17 Recording Runbook	follow a step by step guide from running the app to a finished video, including what to say
18 Run and Test Guide	run the project and check every case, with one command, the automated suites, or by hand
19 SOLID Design	see how the code follows each SOLID principle, with the proof
20 Code Reference	look up every module, class and function (generated from the code's own docstrings)
21 Architecture Diagrams	see the system, one request, the rules, the lifecycle, the AI workflow and the test layers as diagrams
22 Reviewer Guide	judge the work: what to open, how to check every claim, what is still open
23 UI Walkthrough	see every screen of the running app with real screenshots and the rule each one proves